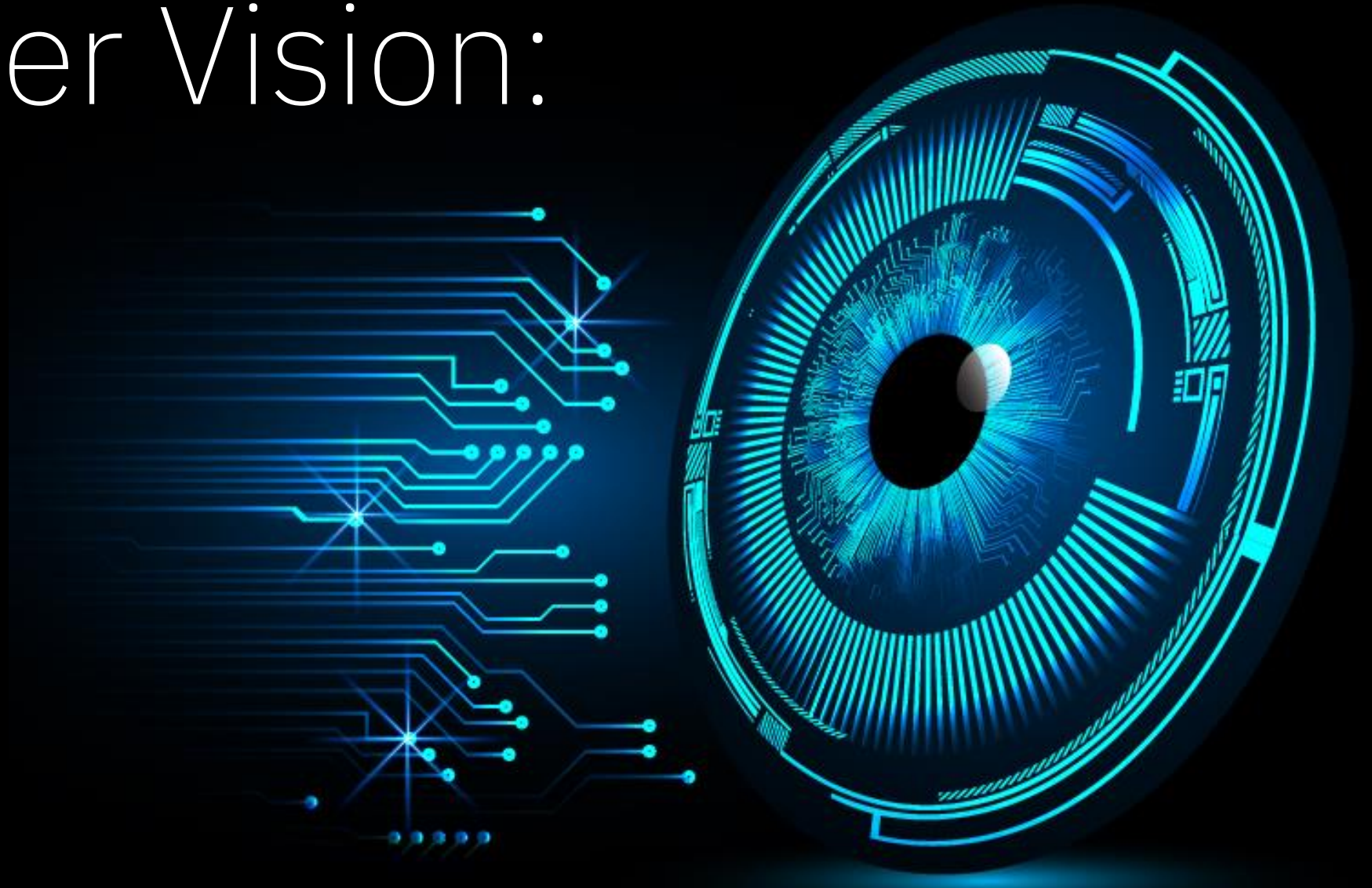


Deep Learning for Computer Vision: CNN



Computer Vision: Explained

Training computer vision models is done in much the same way as teaching children about objects visually.

Instead of a person being shown physical items and having them identified, however, the computer vision algorithms are provided many examples of images that have been tagged with their contents.

In addition to these positive examples, negative examples are also added to the training.

For example, if we're training for images of cars, we may also include negative examples of airplanes, trucks, and even boats.

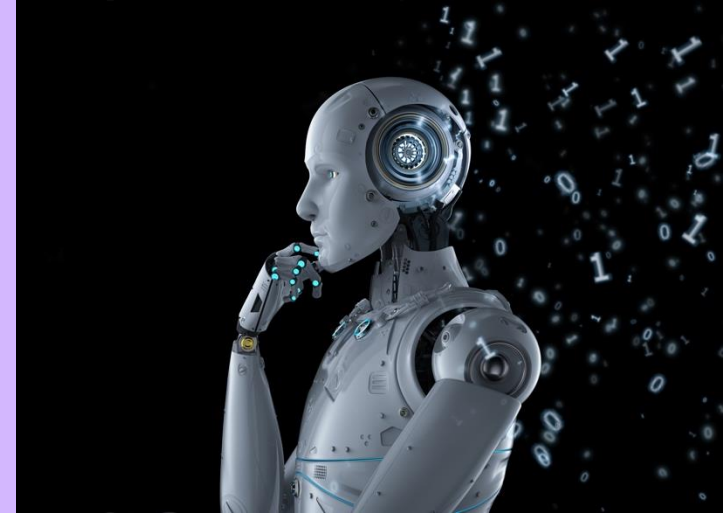


Image Classification and Tagging

Computer vision's most core functionality, *general image tagging and classification*, allows users to understand the content of an image.



"A green bird sitting on top of a bowl"

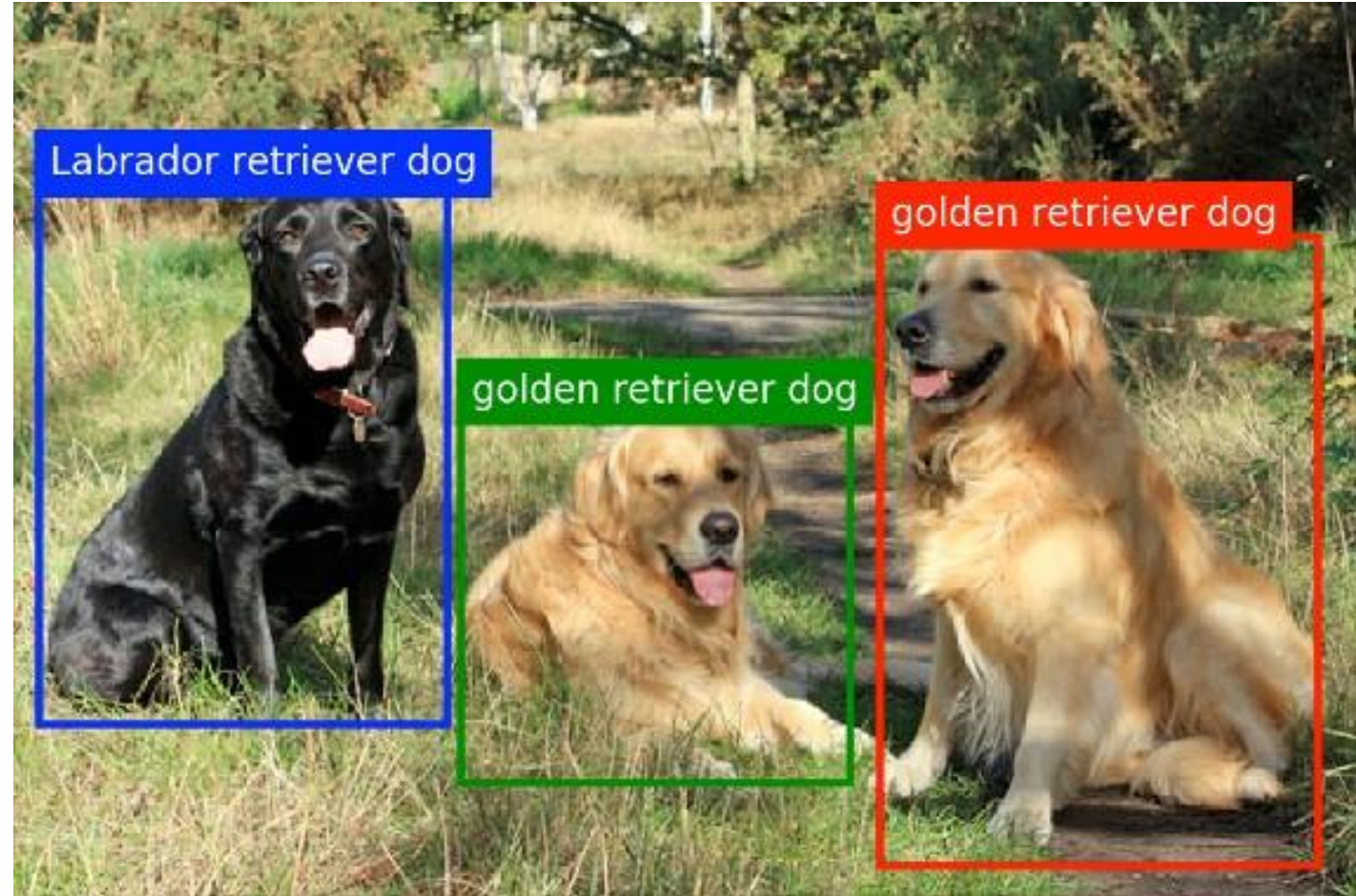
For example

You may need to find images with contents of “male playing soccer outside” or organize images into visual themes such as cars, sports, or fruits.

Object Localization

Sometimes your application's requirements will include not just classifying what is in the image

But also understanding the position of the particular object in relation to everything else



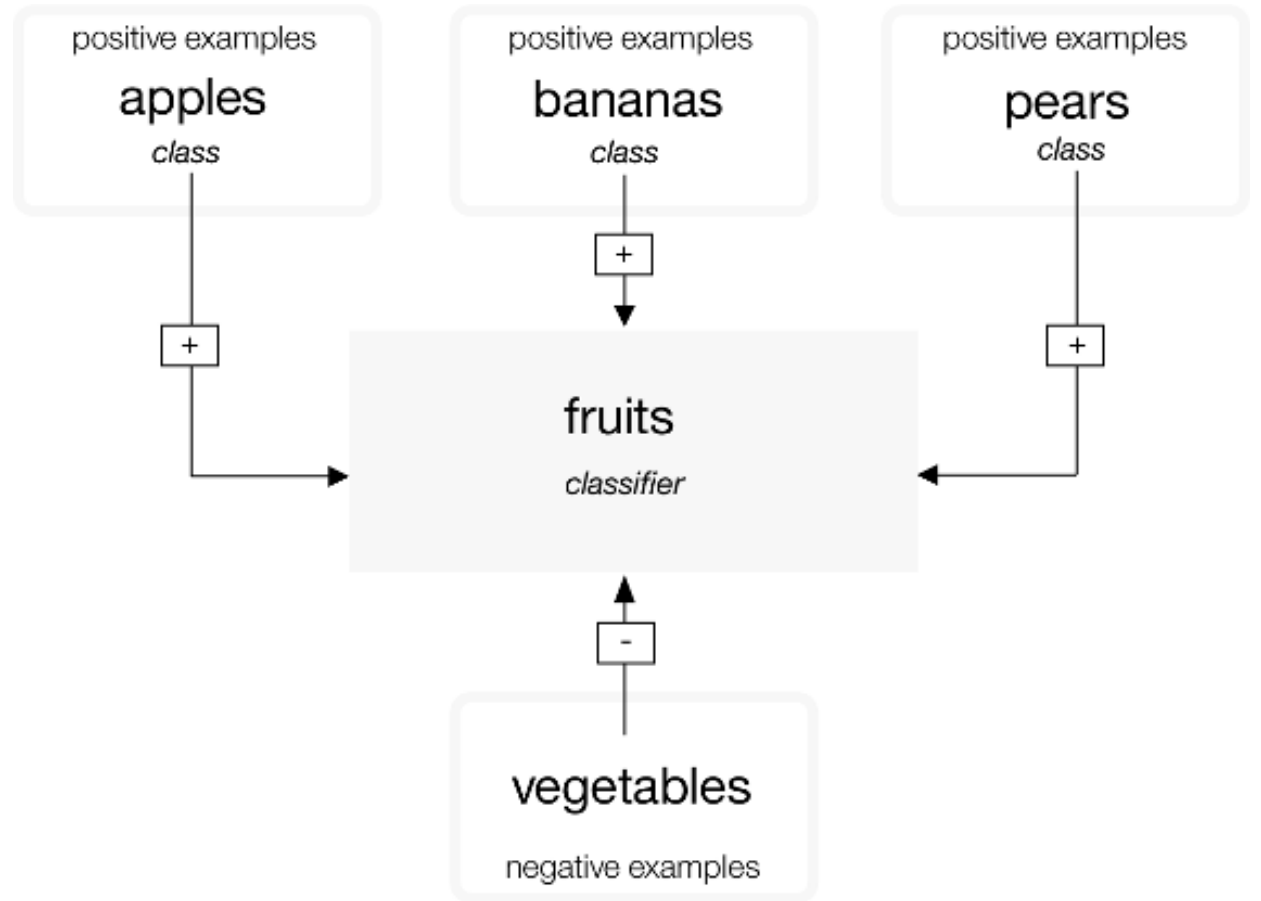
Custom Classifiers

If you're looking to identify or classify only a small set of objects, custom classifiers could be the right tool.

Most of the large third-party platforms provide some mechanism for building custom visual classifiers, allowing you to train the computer vision algorithms to recognize specific content within your images

For example,

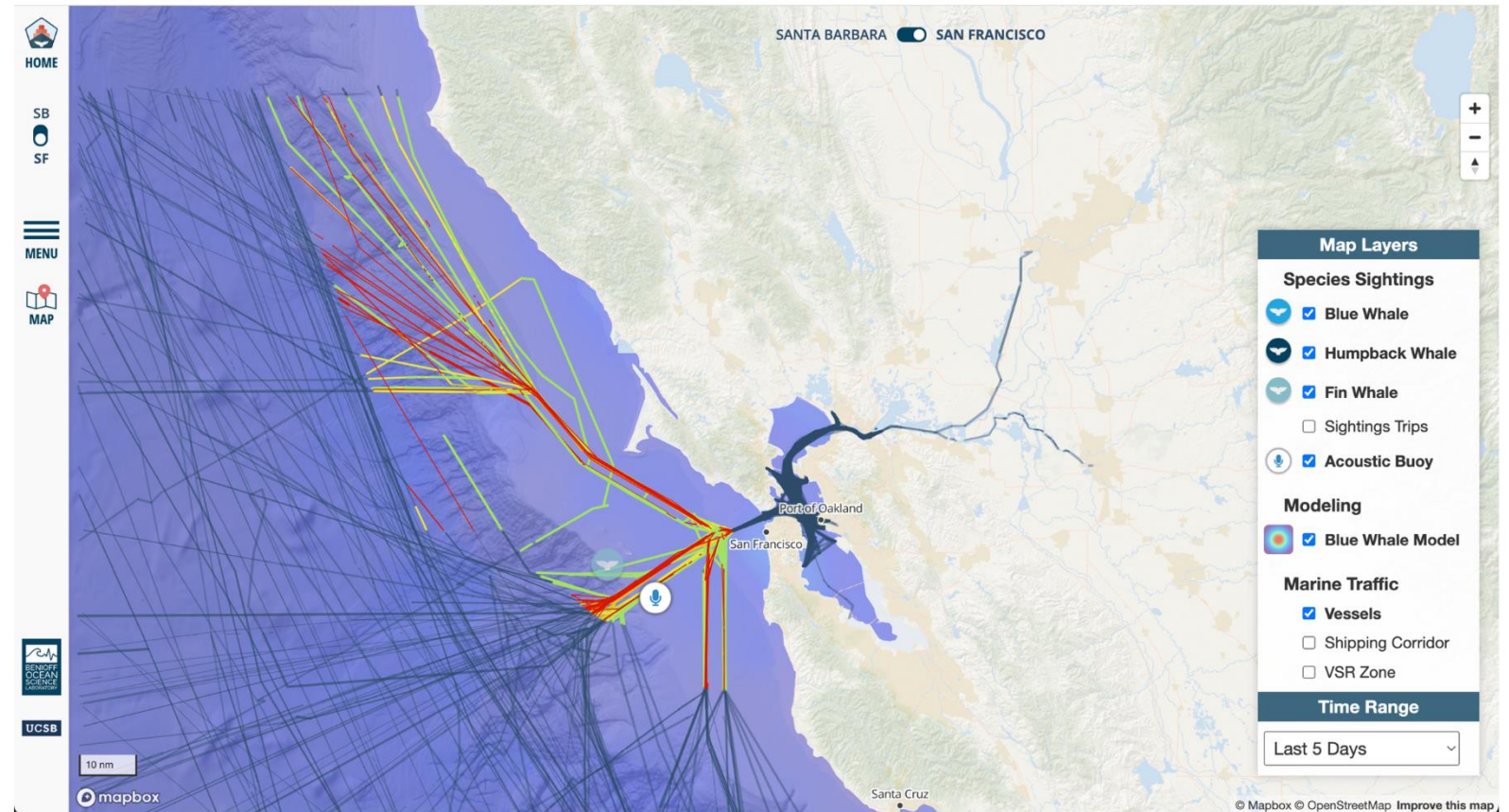
If you were training a custom classifier on fruits, you'd want to have positive training images of apples, bananas, and pears. For negative examples, you could have pictures of vegetables, cheese, or meat.



Use Cases: Satellite Imaging

Whales do not travel the same migratory path from nutrient-rich Antarctica to fertile grounds near the equator anymore.

Changing water temperatures have damaged the coral reef and altered the mating path of other fish that they feed on.



<https://wwfwhales.org/news-stories/whale-trails>

A screen view of the Whale Safe tool by Benioff Ocean Science Laboratory and UCSB. © Mapbox / Open Street Map / Whale Safe

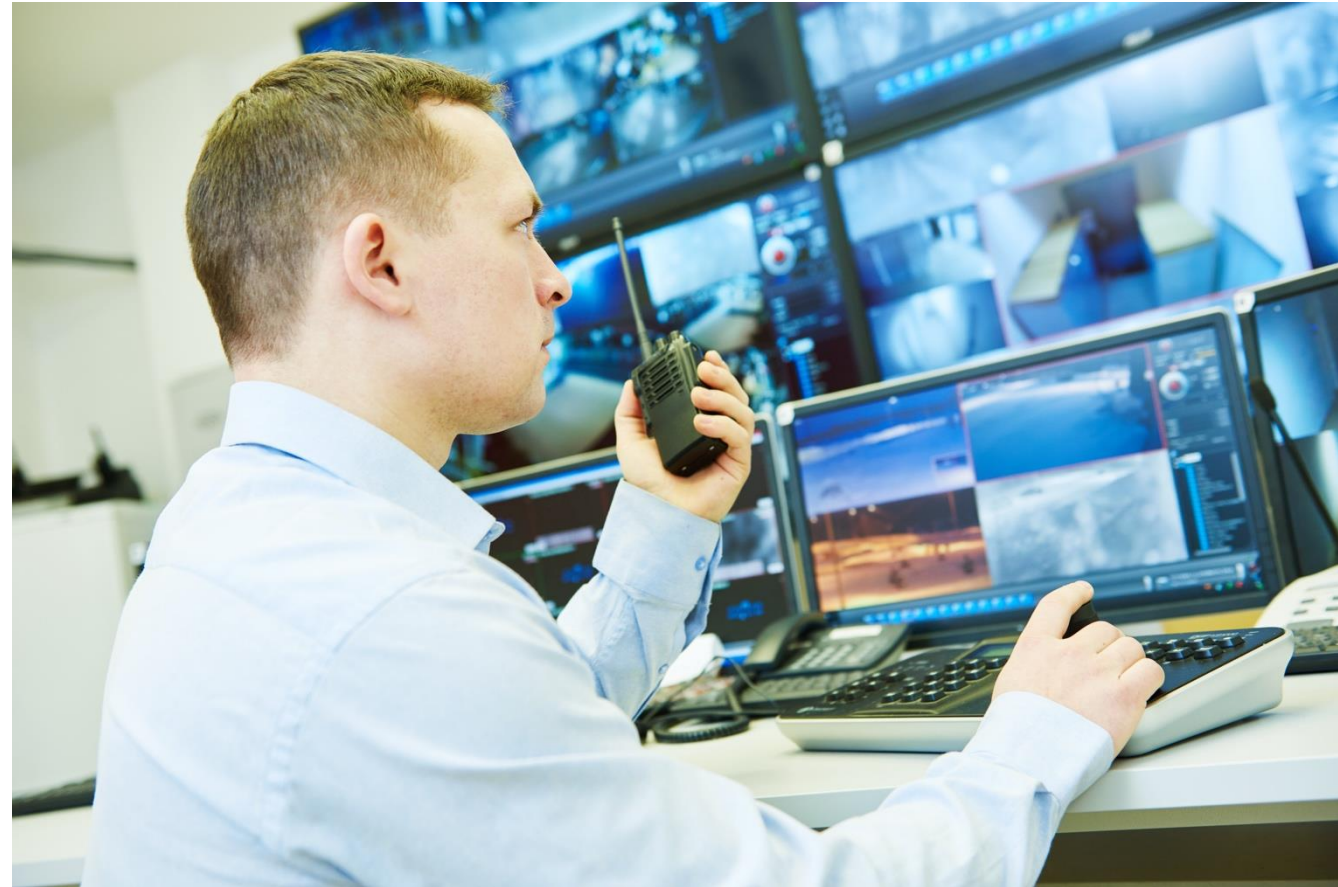
Watch the whales, they know how Earth is changing!

Use Cases: Video Search in Surveillance and Entertainment

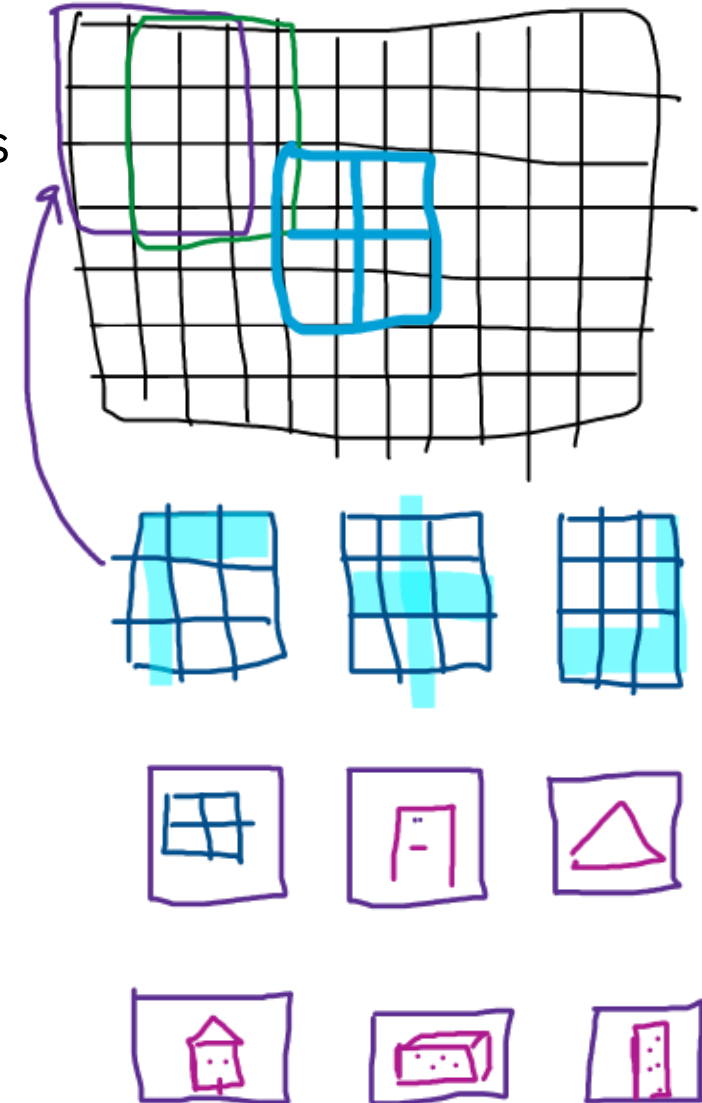
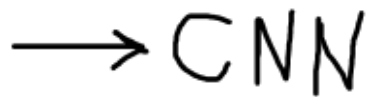
The proliferation of cameras in recent years has led to an explosion in video data

In home surveillance videos, the only viable solution is still human monitoring 24/7, watching screens and raising an alert when something happens.

BlueChasm's product, VideoRecon, can watch and listen to videos, identifying key objects, themes, or events within the footage. It will then tag and timestamp those events, and then return the metadata to the end user

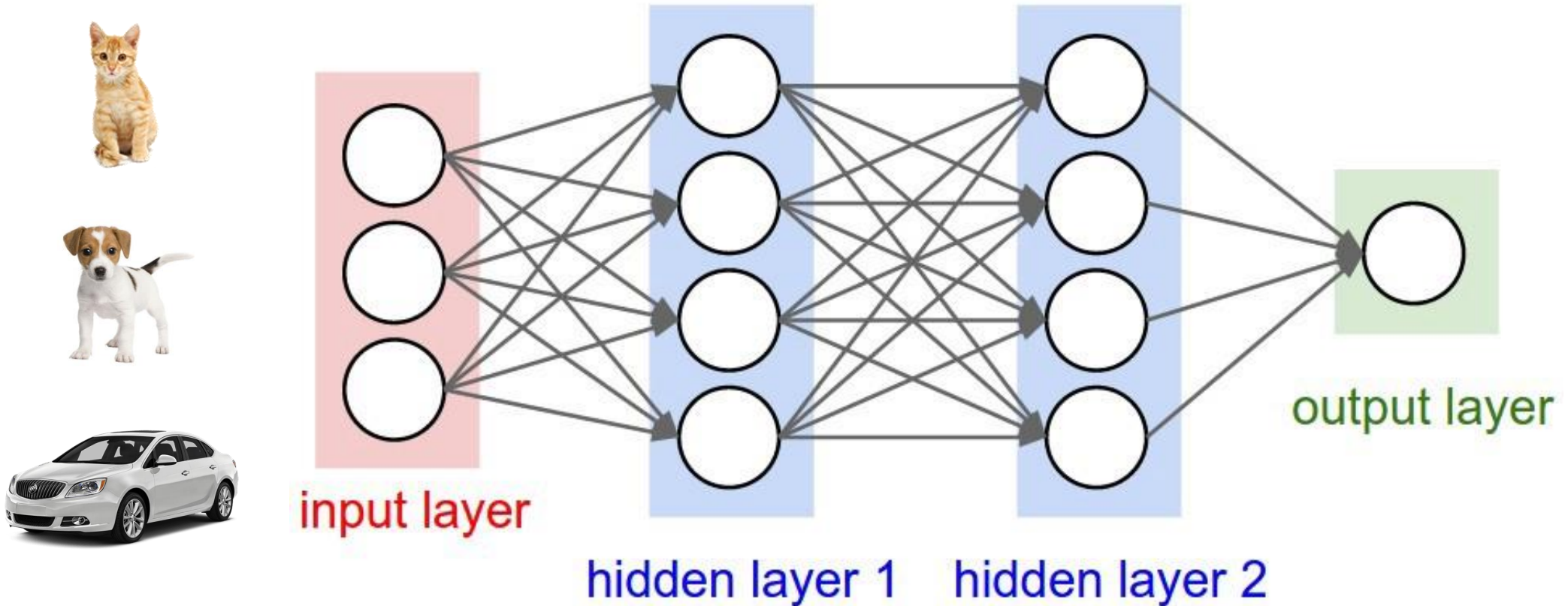


<https://youtu.be/QzY57FaENXg>



How do pixels and features turn into vectors? 1-hot encoding

Labeled input → supervised learning



What if your data comprise images, tweets, videos?



[x,x,x]

[1,0,0]



[x,x,x]

[0,1,0]



[x,x,x]

[0,0,1]



Index or element in a vector

1-hot encoding allows you to encode categorical data into numbers

1-Hot Encoding:

- Create a matrix of 0's and 1's
- Make each category a column in a table
- **Warning:** *you must encode (n-1) categories you have in the variable*
 - Otherwise, you will have *perfect multicollinearity* and encounter mathematical issues

Cons:

- This makes the data very large and sparse
- Better methods for text data
- Does not scale well to big data

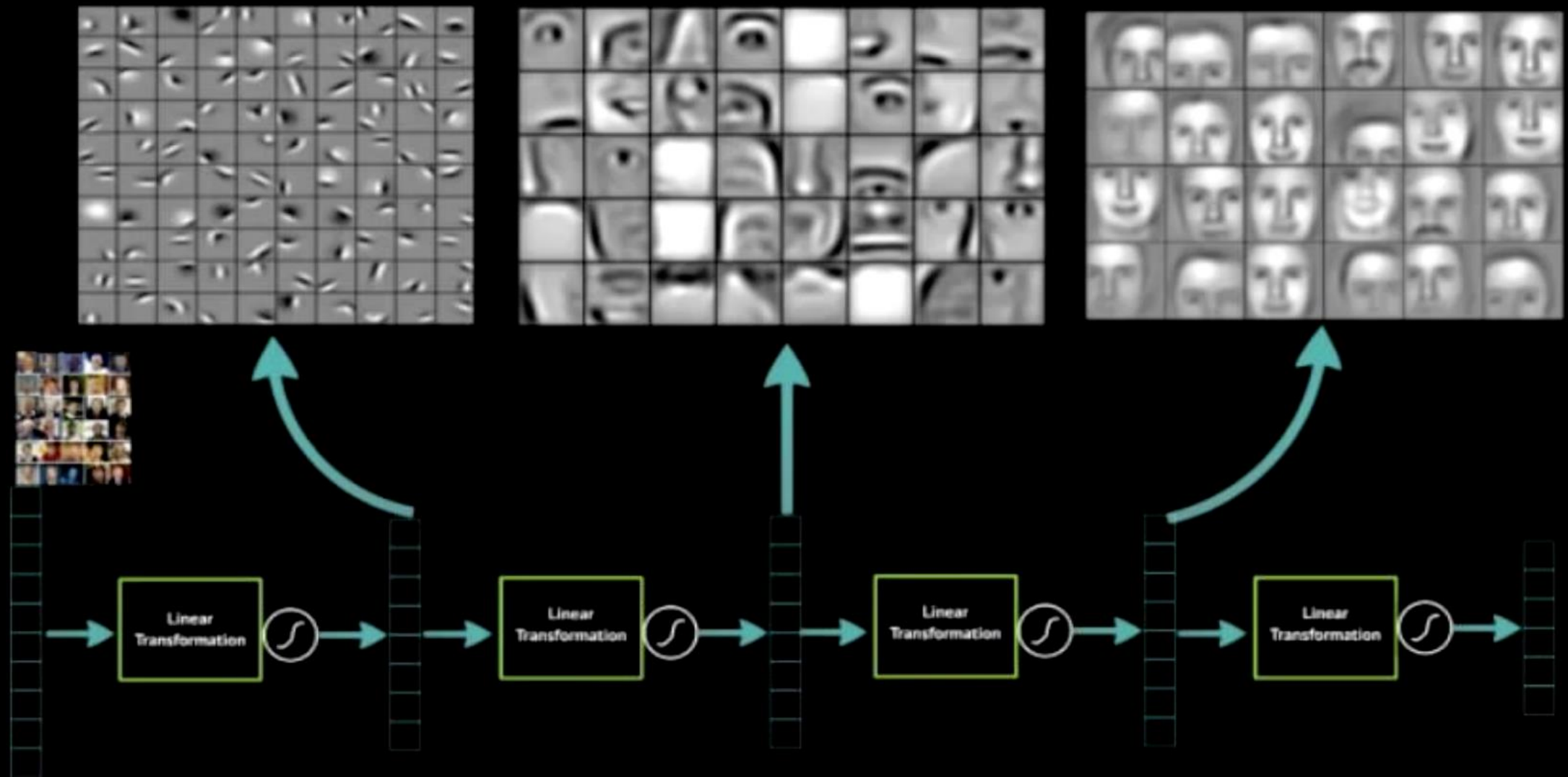
Solutions:

- Bag of words / TF-IDF for text data
- Advanced algorithms such as neural networks with embedding layers

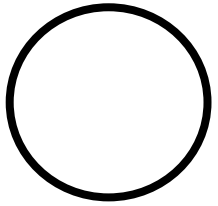
Sample	Species
1	Human
2	Human
3	Penguin
4	Octopus

Sample	Human	Penguin	Octopus
1	1	0	0
2	1	0	0
3	0	1	0
4	0	0	1

Deep Learning learns layers of features



Let's Build-up a Neural Network



Node (neuron)



Connection (Synapse)

$x_1, x_2, x_3, \dots, x_n$

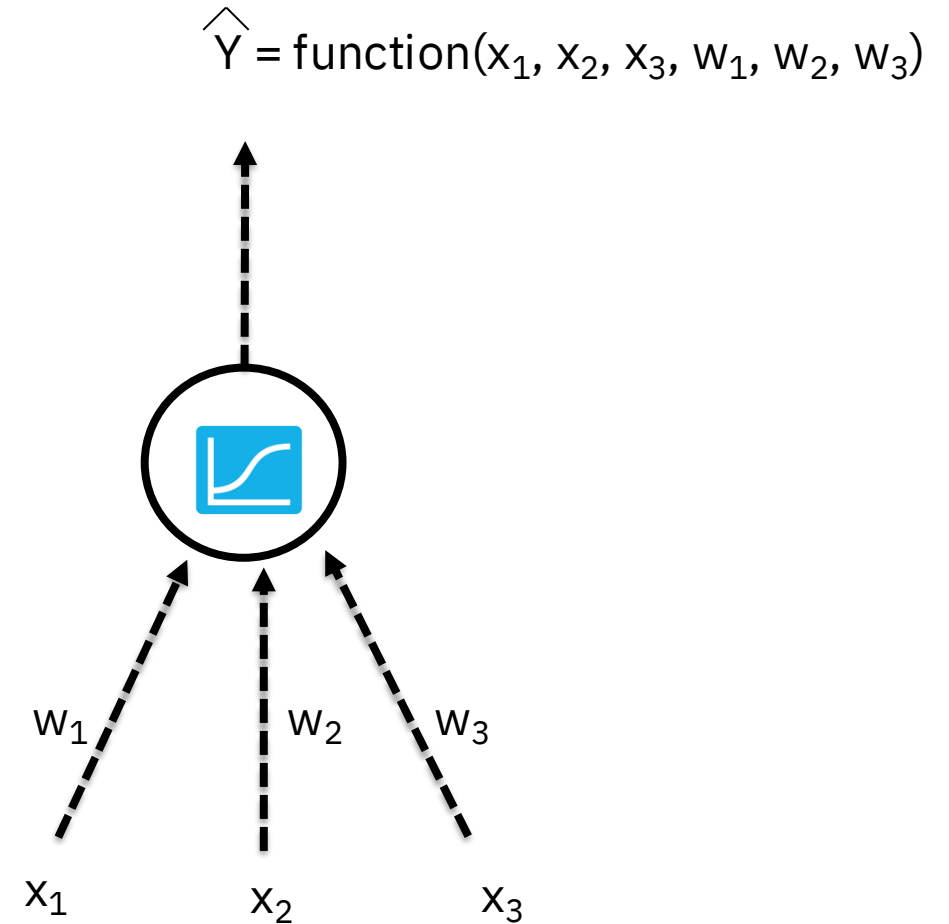
Input

$w_1, w_2, w_3, \dots, w_n$

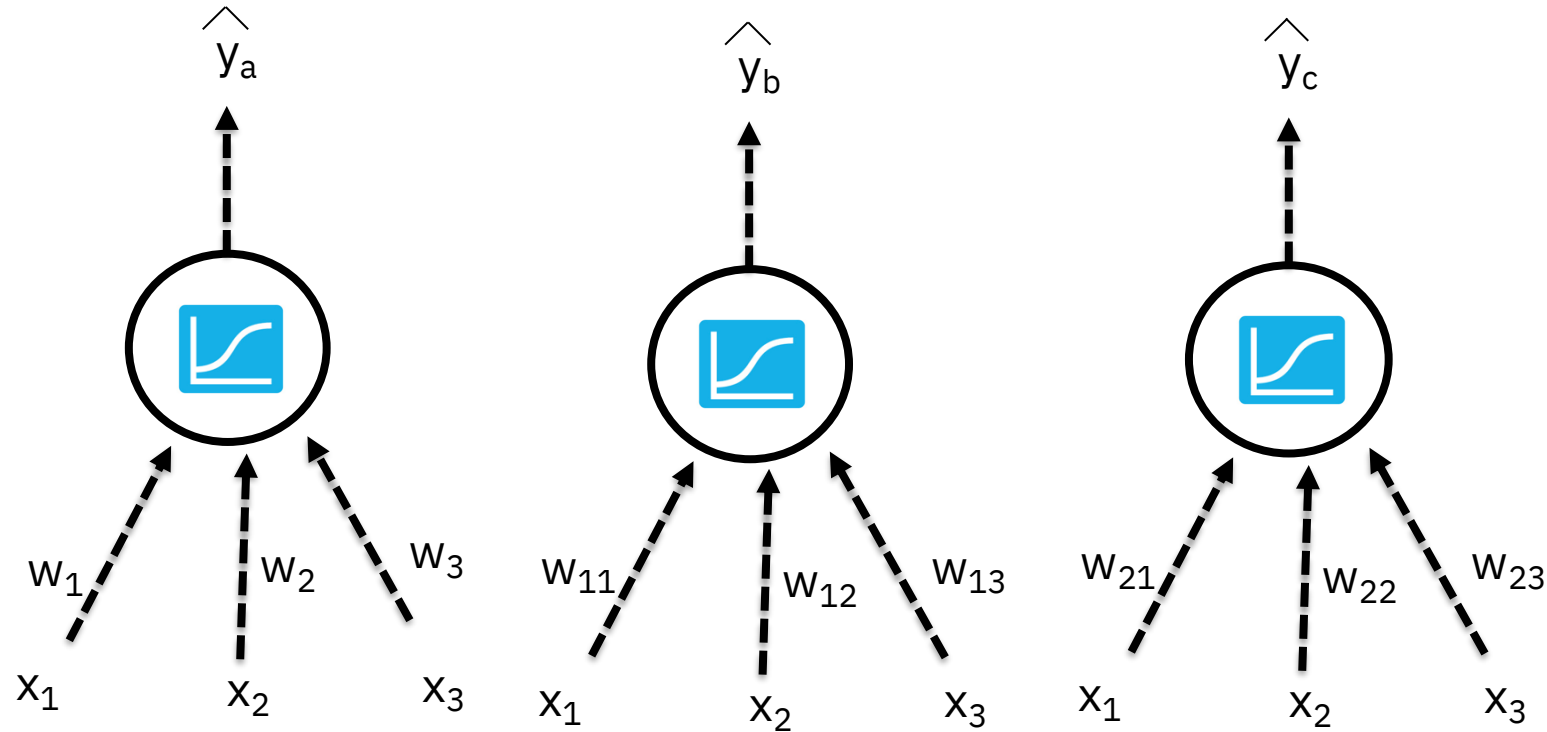
Weights

$\hat{y}$

Predicted output

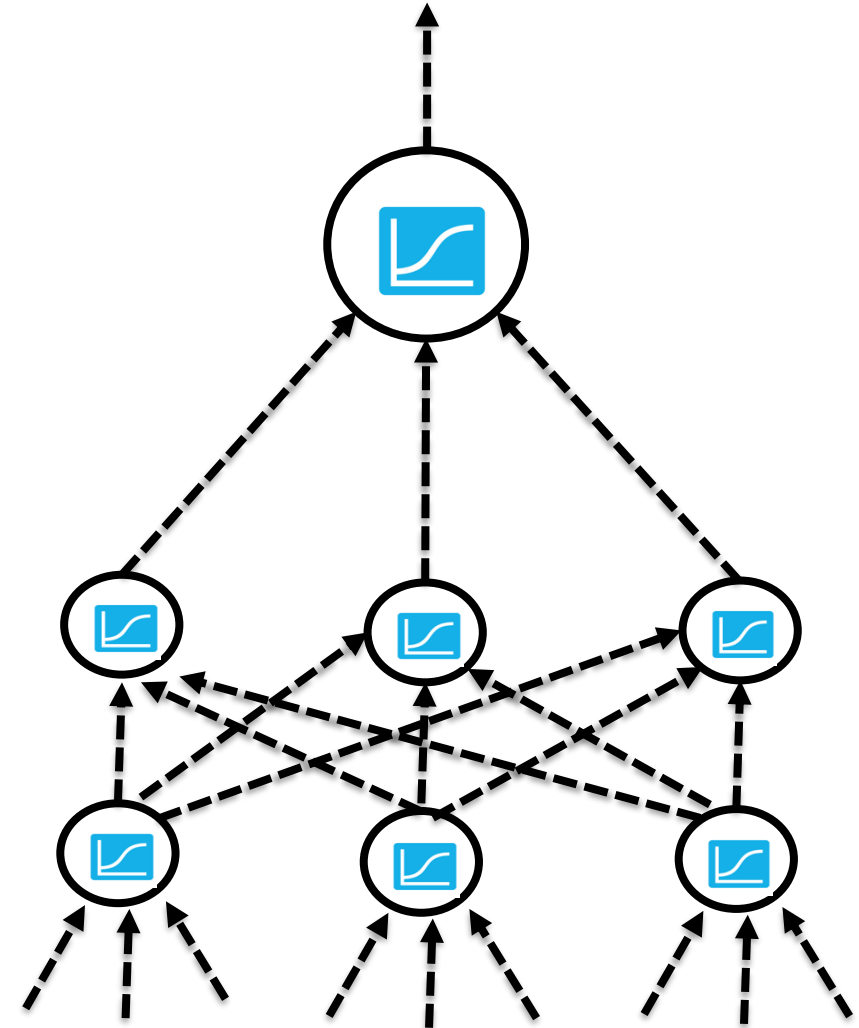
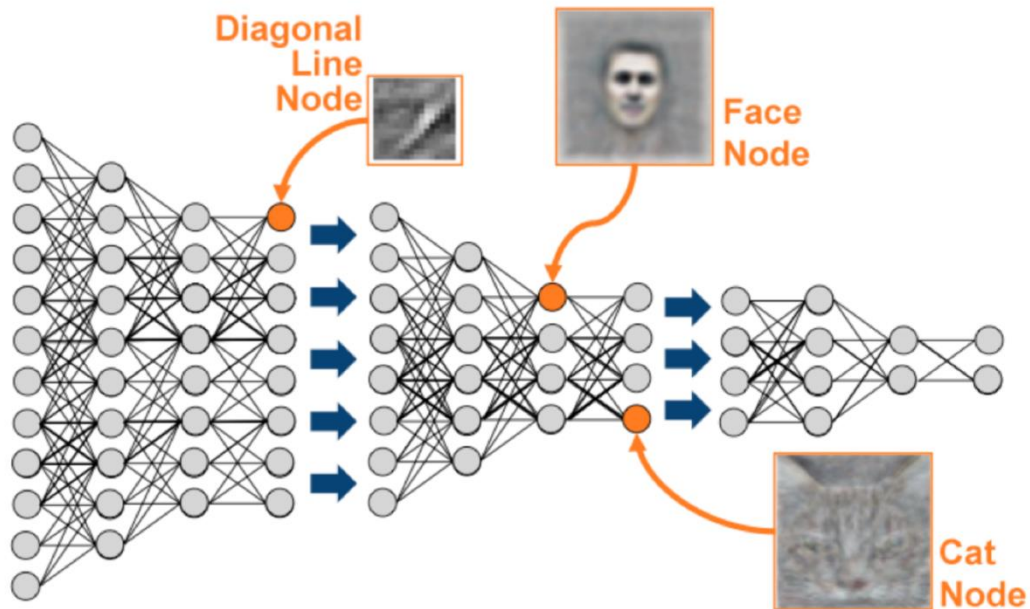


A Layer is a group of nodes organized at the same level

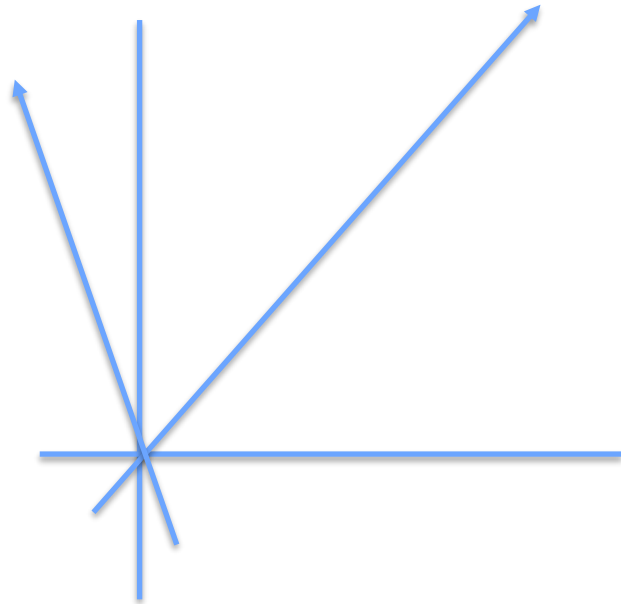


Multiple layers are known as a multiperceptron or deep learning systems

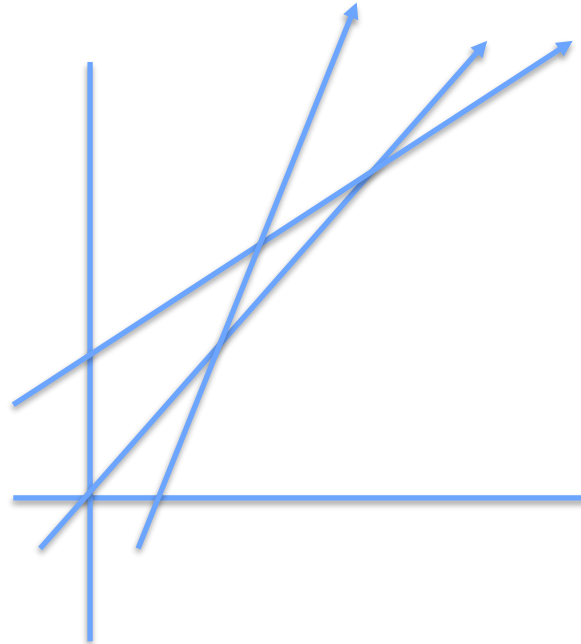
- A network that uses binary classification the output will be two nodes.
- The bias used in the nodes can be a sigmoid operation, parabolic tangent or a Rectify linear unit (ReLU)



Why weights and why bias?

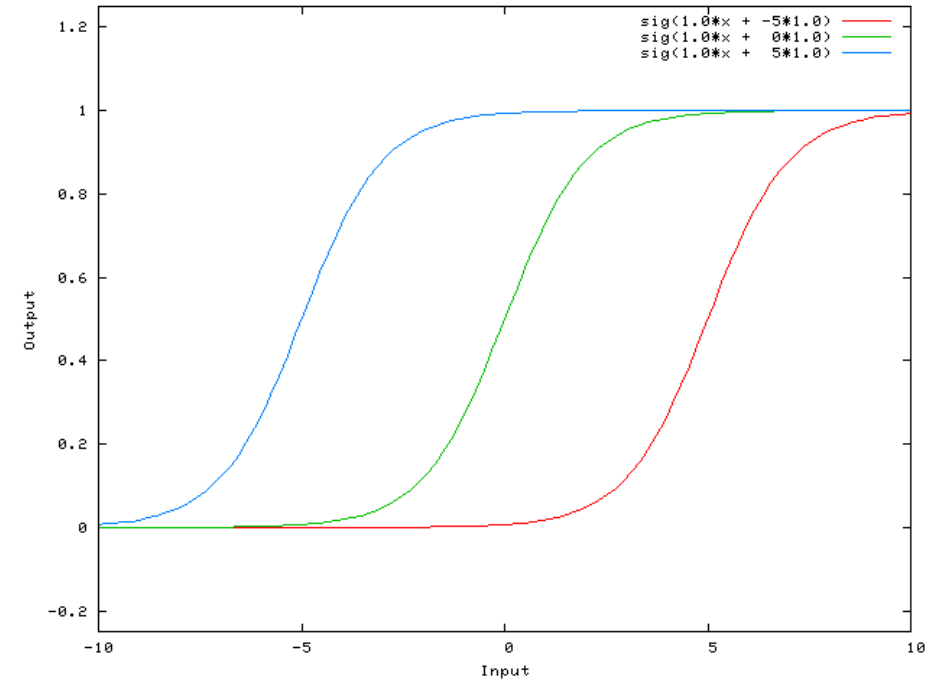


$$y = (f)x$$



$$y = (f)x + b$$

The bias value (b) allows the activations function to be shifted to the left or to the right, to better fit the data; hence, the changes to the weight alter the steepness of the vector or curve. The bias, on the other hand, shifts the entire vector or curve so it fits better.



Three sigmoid curves —
the same input data, but with different biases

Let's talk about:

Learnable Parameters: It learns on its own

...and **Hyperparameters:** humans help the machine learn

What are Learnable Parameters?

These are the **things the model learns on its own** during training.

In a CNN, learnable parameters include:

- Weights of filters/kernels in convolutional layers
They learn *what patterns* (like edges or curves) to detect
- Bias values that help improve flexibility of the model

Analogy:

Learnable parameters are automatically updated using backpropagation and gradient descent as the model trains.

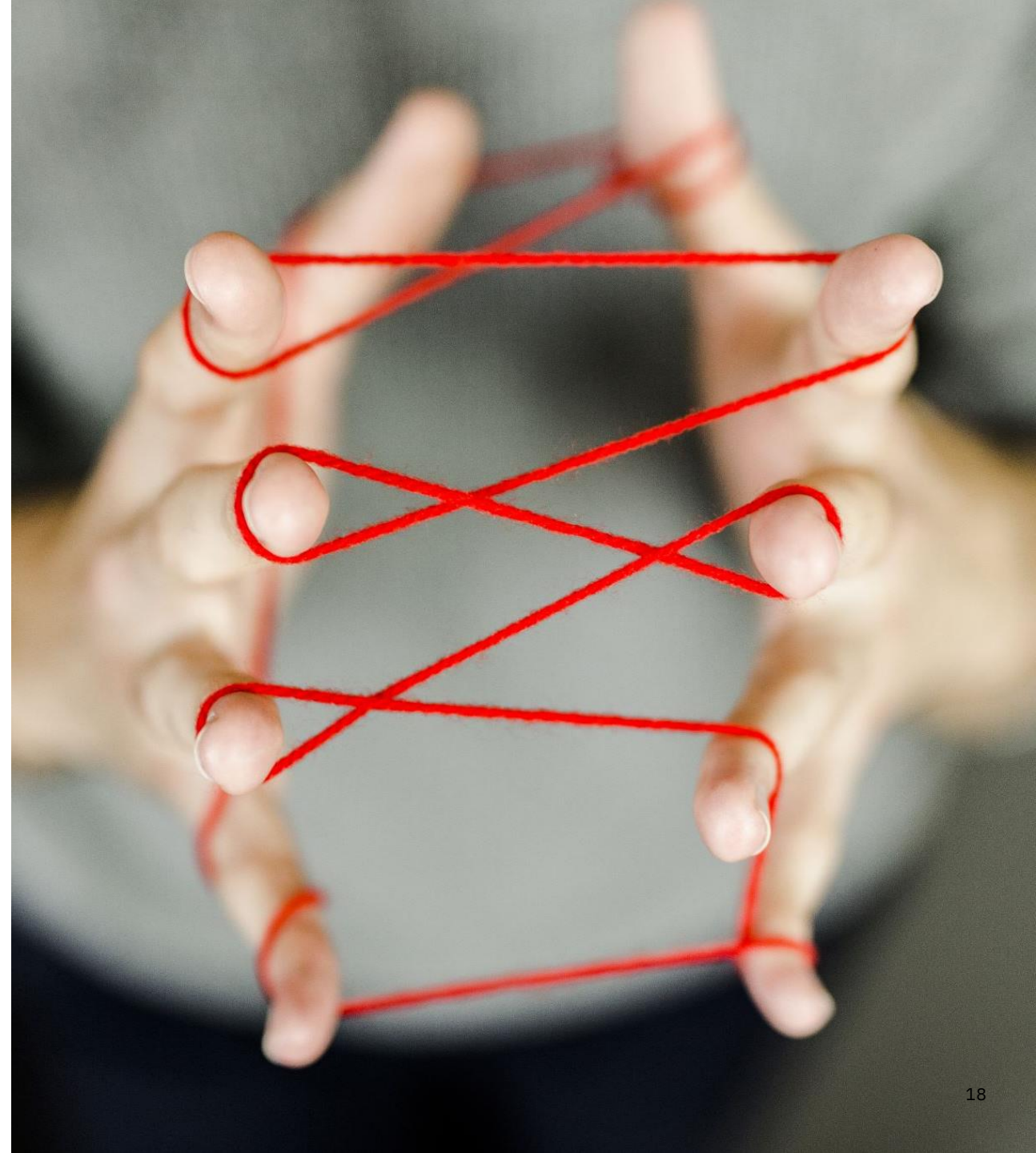


Analogy:

Imagine you're learning to throw a basketball into a hoop.

The way your arm moves (angle, strength) improves with practice.

These are *learned*—like the model learning filter weights.



What are Hyperparameters?

These are the **settings YOU choose before training** the model.

The model doesn't learn these by itself—you set them manually.

Common CNN hyperparameters:

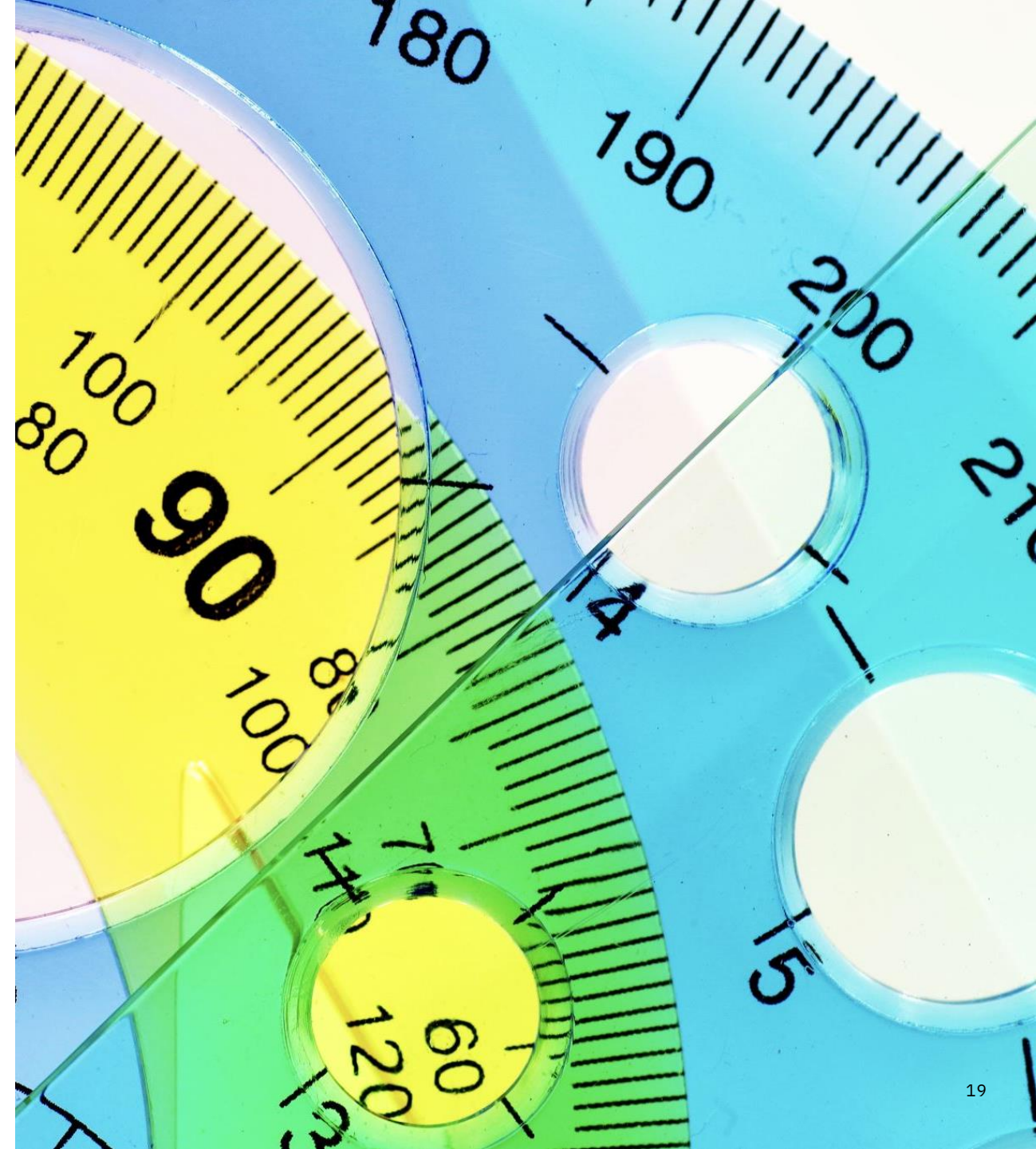
- **Number of filters** (how many features to detect)
- **Filter size** (like 3×3 or 5×5)
- **Learning rate** (how fast the model learns)
- **Batch size** (how many samples to train at once)
- **Number of epochs** (how many times to see the full dataset)
- **Dropout rate** (how much to randomly ignore neurons to avoid overfitting)

Analogy:

In cooking, the ingredients (like spices or oven temp) are **hyperparameters**.

You adjust them before cooking, but the dish "learns" to taste good as it cooks.

If your model is not performing well, tweaking hyperparameters may help!



Details of a CNN

Input Layer: This layer represents the input image itself. For color images, it typically has three channels (red, green, and blue) representing the pixel values. You can visualize this as a 3D matrix where the height and width correspond to the image dimensions, and the depth corresponds to the color channels.

Convolutional Layer: This is the core of a CNN, responsible for feature extraction. It involves applying a set of learnable filters (also known as kernels) to the input image. Each filter is a small matrix of numbers that slides across the image, performing an element-wise multiplication with the part of the image it covers. This process, called convolution, extracts features like edges, textures, or shapes

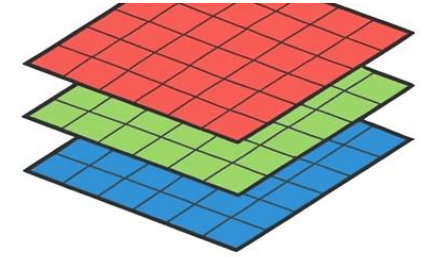
Activation Function (e.g., ReLU): After a convolutional operation, an activation function, commonly ReLU (Rectified Linear Unit), is applied to introduce non-linearity into the network. ReLU sets negative values to zero while keeping positive values unchanged, effectively removing irrelevant features from the activation maps and improving training speed. You can visualize this as a filter "lighting up" only when certain features are strongly present.

Pooling Layer: This layer helps reduce the spatial dimensions of the feature maps, reducing the number of parameters and computations, and making the network more robust to variations in object position. Max pooling, for example, takes the maximum value from a small region of the feature map, effectively summarizing the most prominent features in that area. Imagine taking the brightest spots in different sections of a feature map to create a smaller, more focused representation.

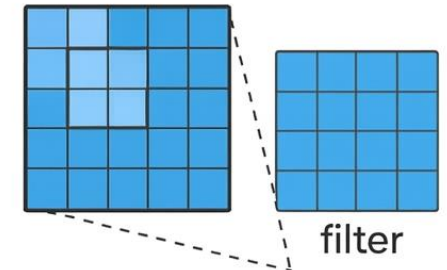
Fully Connected Layer: After several convolutional and pooling layers, the flattened feature maps are fed into fully connected layers. These layers are similar to traditional neural networks, where every node in the layer connects to every node in the previous layer. They learn to combine the extracted features to perform the final classification or prediction task.

Output Layer (e.g., Softmax): The final layer of a CNN for classification tasks typically uses a Softmax activation function. Softmax converts the outputs of the fully connected layer into a probability distribution over the possible classes, where the sum of probabilities for all classes equals 1. The class with the highest probability is then chosen as the predicted class.

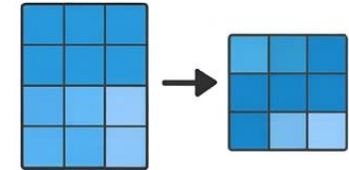
Input Layer



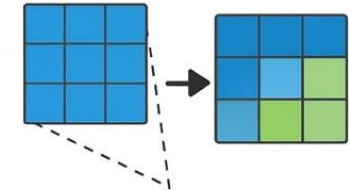
Convolutional Layer



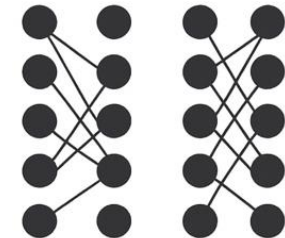
Activation Function (ReLU)

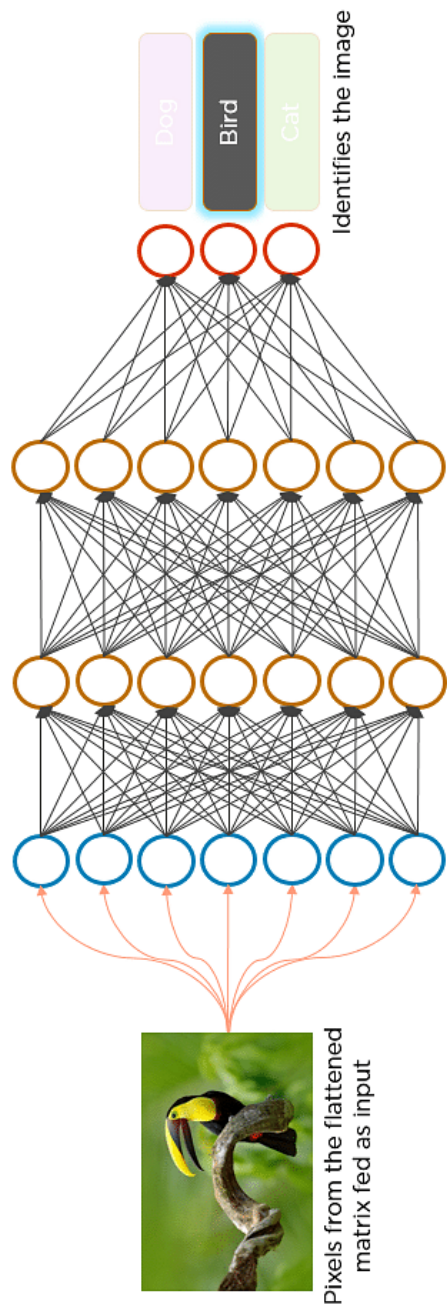


Pooling Layer



Fully Connected Layer





Advantages of CNNs

Learns features automatically

Works great with image data

Reduces number of parameters

Handles large images efficiently

Drawbacks

Needs a lot of labeled data

Training takes time and computing power

Hard to understand what it's learning

May get confused by rotated or unusual images

Gen-AI models are **the new cameras**

Photo and Video
Camera Brands

Nikon

Canon

SONY

RED

ARRI

dji

Panasonic

Generative AI
Image/Video
Brands

Pika

 **Grok**

OpenAI

 **Lightricks**

Google



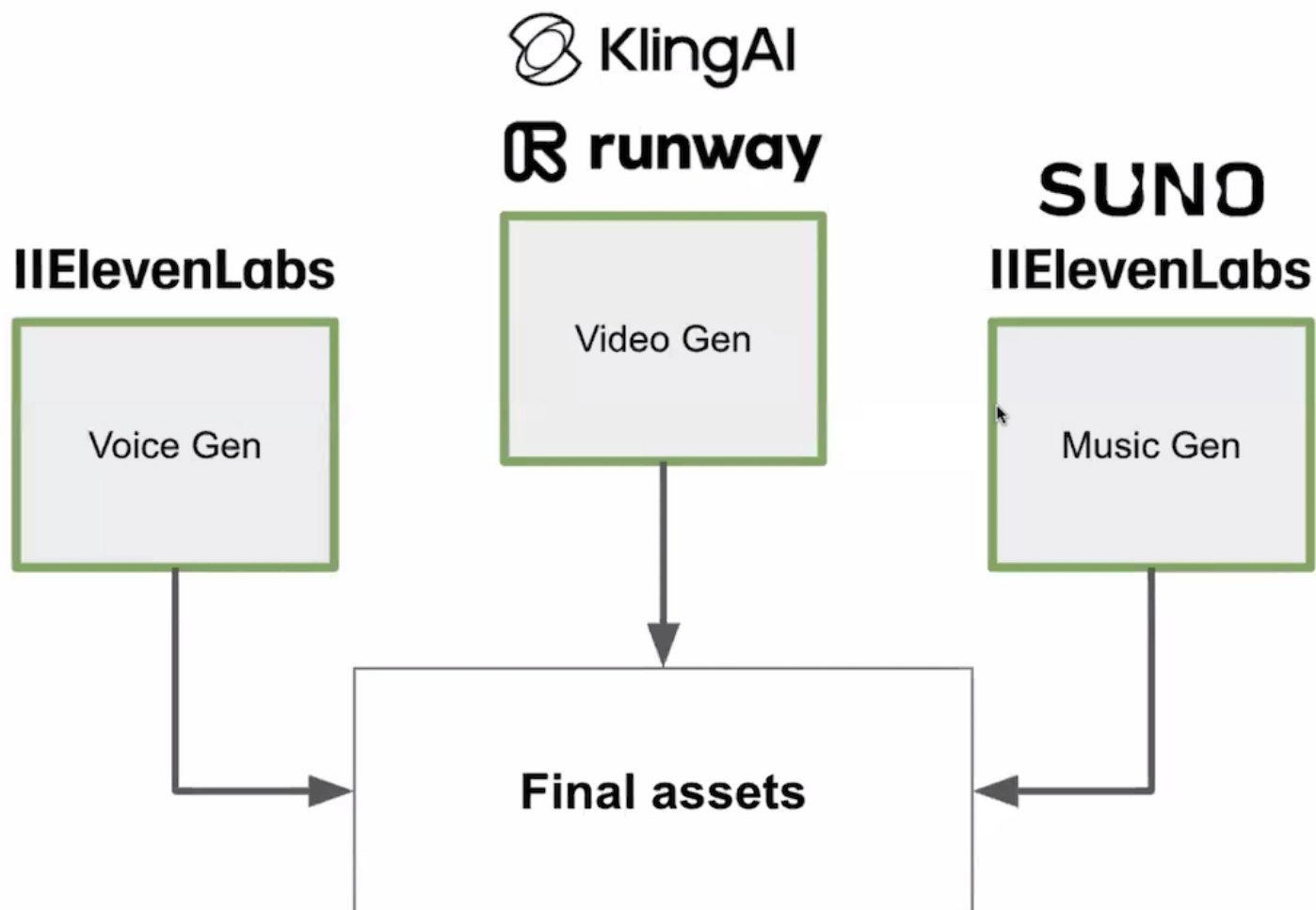
 **ByteDance**

FLUX

 **KlingAI**

 **Hailuo AI**

This is how
you go to
production



	Model	Used for	Modalities	Credits
Alibaba	WAN2.6	Video generation with or without audio	Text to Video Image to Video	720p10/sec 1080p15/sec
	WAN2.6 Flash	Video generation with or without audio	Text to Video Image to Video	720p5/sec 1080p10/sec
	HappyHorse 1.0	General-purpose video generation	Text to Video Image to Video	1080p30/sec 72015/sec
ByteDance	Seedream 5.0-lite	Budget-friendly image generation	Text to Image	2K/3K4/img
	Seedream 5.0 Pro	Precision image generation and editing	Text to Image Image to Image	1K5/img 2K10/img
	Seedance 2.0	Multi-modal video generation with audio	Text to Video Image to Video Video to Video	480p36/sec 720p36/sec 1080p40/ sec4K150/sec
	Seedance 2.0 Fast	Multi-modal video generation with audio, built for speed and exploration	Text to Video Image to Video Video to Video	29/sec
	Seedance 2.0 Mini	Budget friendly video generation, built for speed and exploration	Text to Video Image to Video Video to Video	16/sec
	Seed Audio 1.0	Generate multi-character dialogue, music, and ambient sound simultaneously	Text to Audio Audio to Audio	15/min (5 credit minimum per generation)

	Model	Used for	Modalities	Credits
Google	Veo 3.1	Most advanced video generation with audio and speech	Text to Video Image to Video Start/End Frame	No audio20/sec With audio40/sec
	Veo 3	Previous-gen video generation with audio	Text to Video Image to Video	With audio40/sec
	Nano Banana Pro 2	Most advanced and budget-friendly image generation	Text to Image Image to Image	1K7/img; 2K11/img 4K16/img
	Nano Banana Pro	Most advanced image generation and editing	Text to Image Image to Image	1K/2K20/img; 4K40/img
	Nano Banana	Budget-friendly image generation and editing	Text to Image Image to Image	5/img
	Nano Banana Pro 2 Lite	Fast and budget-friendly image generation	Text to Image Image to Image	4/img
	Gemini Flash 2.5	Well-rounded and budget-friendly LLM for text processing, image analysis, and more in Workflows.	Text Processing Image analysis	1/run
	Lyria 3 Pro	Generating music in Workflows	Text to Audio	8/song
	Lyria 3 Clip	Generating music in Workflows	Text to Audio	4/song

	Model	Used for	Modalities	Credits
Kling	Kling 3.0 Pro	Most advanced video generation with audio and speech	Image to Video; Text to Video; Start/End Frame; Multishot	No audio12/sec With audio17/sec
	Kling 3.0 Std	Budget-friendly video generation	Image to Video; Text to Video; Start/End Frame	No audio9/sec With audio13/sec
	Kling 2.6 Pro	Previous-gen video generation with audio	Image to Video; Text to Video; Start/End Frame; Audio	25/sec
	Kling 2.5 Turbo Pro	Previous-gen budget-friendly video generation	Start/End Frame	720p12/sec 1080p15/sec
	Kling 2.5 Turbo	Previous-gen budget-friendly video generation	Image to Video	10/sec
	Kling 3.0 Motion Control	Character animation with a performance video	Character performance	1080p17/sec 720p13/sec
	Kling O3	Multi-modal video generation with audio	Text to Video; Image to Video; Keyframes; Multi-shot	4K43/sec; Pro17/sec; Standard13/sec
	Kling 3.0 Turbo	Budget friendly video generation built for speed and exploration	Text to Video Image to Video	720p14/sec; 1080p16/sec

	Model	Used for	Modalities	Credits
OpenAI	GPT Image 1.5	Advanced image generation	Text to Image Image to Image	1024×102414/img Other20/img
	GPT Image 1 Mini	Budget-friendly image generation	Text to Image Image to Image	2/img
	GPT Image 2	Image generation with flexible balance of budget and quality	Text to Image Image to Image	1K & 2K (low quality)1/img 1K & 2K (medium quality)5/img 1K & 2K (high quality)20/img 4K (low quality)2/img 4K (medium quality)11/img 4K (high quality)41/img
Black Forest Labs	FLUX.2 Max	Advanced image generation	Text to Image Image to Image	60010/img1K10/img2K13/img4K19/img
	FLUX.2 Klein	Budget-friendly image generation	Text to Image Image to Image	6004/img1K4/img2K5/img

	Model	Used for	Modalities	Credits
Topaz AI	Upscale to 4K	Upscaling video to 4K	Upscale video	2/sec
Anthropic	Claude	Well-rounded and budget-friendly LLM for text processing, image analysis, and more in Workflows.	Text processing Image analysis	Sonnet 4.53/run Opus 4.54/run Opus 4.84/run
Magnific	Precision	Flexible, customizable image upscaling	Upscale	4k150/img ≤ 4k25/img
	Style Transfer	Transform an image by applying various artistic styles	image to image	15/img

	Model	Used for	Modalities	Credits
xAI	Grok Imagine	Fast, budget-friendly image generation	Text to Image Image to Image	1K2/img; 2K7/img
	Grok Image Quality	Realism and precision for text-heavy, detailed images	Text to Image Image to Image	1K7/img; 2K8/img
	Grok Imagine	Fast, budget-friendly video generation with audio	Text to Video Image to Video	480p13/sec + 1 credit per reference frame 720p17/sec + 1 credit per reference frame
	Grok Imagine	fast, budget-friendly video editing and extending	Video to Video	6/sec
	Grok Imagine 1.5	Fast, budget-friendly video generation with audio	Image to Video	480p10/sec + 1 credit per reference frame 720p16/sec + 1 credit per reference frame
ElevenLabs	Text to Speech	Turn typed text into speech	Text to Audio	1/50 characters
	Text to SFX	Turn typed text into sound effects	Text to Audio	1/6 seconds
	Voice dubbing	Translate recorded speech into a different language	Audio to Audio	1/2 seconds
	Voice isolation	Isolate and remove noise from recorded speech	Audio to Audio	1/6 seconds
	Speech to Speech	Transform recorded speech into a new voice	Audio to Audio	1/3 seconds